claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-af0bb9cff54b9e27b.jsonl`
- SHA-256: `38a254dda32106fbe0b9ab6b3bdc9c1455535d2c1e0dcf0cd014b74dcc963575`
- Source modified: `2026-06-18T07:54:30+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e5140909-890`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T07:52:17.088Z)

```
Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong
logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch,
unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix +
confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.

CANDIDATE:
{
  "title": "FusionSegmenter axis-estimate cache keyed on id(points) can return a stale play-axis
estimate across videos",
  "file": "backend/pipeline/segmenters/fusion_hybrid.py",
  "line": "100-109 (_axis_estimate, key = id(points))",
  "trigger": "Reusing one FusionSegmenter instance for two process_video calls where the second
call's `points` list is allocated at the same memory address a prior (freed) trajectory list
held ? CPython readily reuses freed list addresses. _axis_cache_key still equals id(points) so
the cached (axis, net, span) from the previous video is returned instead of re-estimating.",
  "why_wrong": "id() is reused after garbage collection, so id()-as-cache-key is unsound across
object lifetimes. A stale net/axis estimate silently mis-scores net_crossings (wrong deadband
and net position) for the new video, which can flip Gate-A keep/drop decisions. Within a single
process_video it is safe (points stays alive), so impact is limited to instance reuse.",
  "severity": "low",
  "proposed_fix": "Key the cache on a content fingerprint, pass the estimate explicitly through
the enrich call chain, or reset self._axis_cache_key/_axis_cache at the start of each
process_video; do not rely on id() identity across calls.",
  "confidence": 0.3,
  "area": "backend/pipeline/segmenters/ + stitcher/ + validators/"
}

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-
base/backend/pipeline/segmenters/fusion_hybrid.py + related test. Two lenses below; you are:
REACHABILITY ? prove a real path hits it or set real=false.
```

2. assistant (2026-06-18T07:52:19.449Z)

I'll analyze this candidate bug about the `id(points)` cache key in `FusionSegmenter`. Let me read the actual code first.

3. user (2026-06-18T07:52:20.354Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
```

```
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
```

```
54          @property
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
70                      "queue_length": cfg.queue_length,
71                      "fusion_substrate": "tracknet",
72                  }
73              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                      frame_width: float, video_path: str,
78                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82              # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87              fcfg = idx_cfg.get("fusion", {})
88              if fcfg.get("cue_flags", {}).get("person", False):
89                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90              return stats
91
92          # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93          # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94          # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95          # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96          # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97          # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98          # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100         def _axis_estimate(self, points: List[Any]) -> tuple:
101             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
```

```
            computed once per video, not once per candidate window (it depends only on
points)."""
            key = id(points)
            if self._axis_cache_key != key or self._axis_cache is None:
                self._axis_cache_key = key
                self._axis_cache = rally_gate.estimate_play_axis(points)
            est = self._axis_cache
            assert est is not None
            return est

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"]
            # If we can't trust the frame dims, every normalised feature would silently
collapse
            # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
            if fw <= 0 or fh <= 0:
                return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

            poly = fcfg.get("court_polygon")
            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                              court_polygon=polygon, net=net)

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
            (0) keep every window ? identical to the substrate. Persisted candidates are
            untouched (full superset).

            NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
            thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
            where no segment clears ``min_crossings``), the gate falls back to the full
ungated
            set rather than emit a video with zero rallies. This makes Gate A/B strictly
            *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
            the substrate is "do no harm": the bounded-windowing / largest-internal-gap
```

```
split
154             below it always wins on an otherwise-empty video. Observed on real footage
155             (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156             gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157             fcfg = idx_cfg.get("fusion", {})
158             min_crossings = int(fcfg.get("min_crossings", 0))
159             min_players = int(fcfg.get("min_players", 0))
160             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162             keep: List[int] = []
163             for i, st in enumerate(window_stats):
164                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                     continue
166                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                     continue
168                 keep.append(i)
169
170             if not keep and window_stats:
171                 # Gating emptied a non-empty video ? fall back to the ungated set.
172                 return list(range(len(window_stats)))
173             return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177
```

4. assistant (2026-06-18T07:52:24.534Z)

Now let me understand the call chain. The key question for REACHABILITY: is `FusionSegmenter` instance reused across multiple `process_video` calls? Let me trace where `_enrich_candidate_stats` / `_axis_estimate` are called and how the segmenter instance is created/lifecycle'd.

5. user (2026-06-18T07:52:25.749Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
```

```
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry
    from backend.pipeline.detectors.base import DetectorRunner

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras)."""
            raise NotImplementedError

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
```

```
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                          player_pool: Optional[List[str]] = None,
                          max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.get("indexing", {})
            # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
            # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
            # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
            # measure it against the Gemini path, or to avoid the cloud entirely). With
            # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
            skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            if skip_ai:
                model_name = "local-noai"
                logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
                            "be emitted as rallies; no %s handoff, no API key needed.",
                            label, provider_name)
            else:
                model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                    )
                    return []
```

```
                try:
                    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            runner, runner_params = self._build_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.get("max_window_duration", 0.0)
            window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
            window_hard_cap = idx_cfg.get("window_hard_cap", False)
            window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
            inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
            inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
            window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
            window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s WSL environment not ready: %s", label, msg)
                return []
            logger.info("%s WSL env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
```

```
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
                max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)

            ai_segments: List[dict] = []
            if skip_ai:
                # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
                # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
                # falls back to the duration-based estimate (no AI exchange count).
                ai_segments = [
                    {
```

```
                        "start_time": all_candidate_windows[wi]["start"],
                        "end_time": all_candidate_windows[wi]["end"],
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
                            "(no AI handoff).", len(ai_segments))
            else:
                handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info("Phase 3: %s validation on %d candidate windows...",
                            provider_name, len(handoff_indices))

                def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                    w_start = max(0, window["start"] - handoff_padding)
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                        return []
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
                                                         label=f"Handoff {wi+1}")
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                    futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                               for wi in handoff_indices]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
                logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
                    start = float(segment["start_time"])
                    end = float(segment["end_time"])
```

```
319                     dur = end - start
320                     metadata = {
321                         "shot_count": self._shot_count_from(segment, dur),
322                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
323                         "detector": f"{self.family}-hybrid-{model_name}",
324                     }
325                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
326                     if segment_id:
327                         candidate_id = segment.get("_candidate_id")
328                         if candidate_id is not None:
329                             self.db.link_candidate_to_segment(candidate_id, segment_id)
330                         segments_data.append({
331                             "id": segment_id, "start_time": start, "end_time": end,
332                             "duration": dur, "metadata": metadata,
333                         })
334                 except Exception as e:
335                     logger.warning("Skipping invalid segment struct: %s", e)
336
337             return segments_data
338
```

6. assistant (2026-06-18T07:52:30.746Z)

Key observation: within a single `process_video`, `points` is bound at line 202 (`points = runner.parse_trajectory_csv(csv_path)`) and stays alive through the `window_stats` comprehension at line 230-233. So within one call, `id(points)` is stable and the cache is safe.

The candidate's claim is about **instance reuse across `process_video` calls**. Let me verify two things:
1. Is a `FusionSegmenter` instance ever reused for multiple `process_video` calls?
2. For the stale-cache bug to manifest, the second call's `points` must get the same `id()` AND the cache must not have been invalidated.

Let me look at how segmenters are instantiated and whether instances are reused.

7. user (2026-06-18T07:52:32.027Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
```

```
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

8. assistant (2026-06-18T07:52:34.335Z)

The registry returns classes (`Type[BasePlaySegmenter]`), not instances. So callers must instantiate. Let me find where `process_video` is called and how the segmenter is instantiated.

9. user (2026-06-18T07:52:35.856Z)

```
scripts\run_process_and_stitch.py:76:        segmenter_cls = segmenter_registry.get(seg_name)
scripts\run_process_and_stitch.py:82:        # process_video returns a Result (Success|Failure)
for some segmenters (e.g. `motion`);
scripts\run_process_and_stitch.py:85:        segments, failure =
unwrap_or_failure(segmenter.process_video(video_id, video_path))
tools\generate_highlights.py:99:    seg_cls = segmenter_registry.get(segmenter)
tools\generate_highlights.py:167:            result = seg_cls(db, cast(Dict[str, Any],
cfg)).process_video(vid, proc_detect)
tests\test_api_endpoints.py:55:    def process_video(self, video_id, video_path, *a, **k):
tests\test_async_jobs.py:63:    def process_video(self, video_id, video_path, *a, **k):
tests\test_async_jobs.py:72:    def process_video(self, video_id, video_path, *a, **k):
tests\test_async_jobs.py:170:        def process_video(self, video_id, video_path, *a, **k):
scripts\run_phase3_eval.py:101:        seg_cls = segmenter_registry.get(args.segmenter)
scripts\run_phase3_eval.py:146:                # process_video returns a Result
(Success|Failure) for some segmenters
scripts\run_phase3_eval.py:150:                    segmenter.process_video(video_id, video_path,
```

```
max_frames=args.max_frames))
tests\test_base.py:15:    def process_video(self, video_id, video_path, player_pool=None,
max_frames=None) -> Result:
tests\test_base.py:34:    success = inst.process_video("v1", "path")
tests\test_base.py:40:    failure = inst.process_video("v1", "fail")
tests\test_base.py:82:        def process_video(self, video_id, video_path, player_pool=None,
max_frames=None) -> Result:
tests\test_base.py:108:        def process_video(self, video_id, video_path, player_pool=None,
max_frames=None) -> Result:
tests\test_base.py:166:    Demonstrates that a raising segmenter in process_video propagates the
exception
tests\test_base.py:187:        def process_video(self, video_id, video_path, player_pool=None,
max_frames=None) -> Result:
tests\test_base.py:192:        inst.process_video("v1", "path")
backend\main.py:170:        segmenter_cls = segmenter_registry.get(seg_name)
backend\main.py:192:            result = segmenter.process_video(video_id, req.video_path,
req.player_pool, req.max_frames)
backend\main.py:347:def process_video(req: ProcessRequest):
backend\main.py:362:    if req.segmenter and not segmenter_registry.get(req.segmenter):
backend\main.py:826:            segmenter_cls = segmenter_registry.get(seg_name)
backend\main.py:835:            result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
backend\eval\fusion_golden.py:70:    axis, net_px, _span = rally_gate.estimate_play_axis(points)
backend\results.py:72:    that forget it (e.g. ``len(process_video(...))``) crash with
``TypeError: ... has no len()``
backend\eval\windowing.py:7:(``TrajectoryHybridSegmenter.process_video``) windows COARSELY
(``vt=0.02, merge_gap=2.0,
backend\eval\windowing.py:105:    ``TrajectoryHybridSegmenter.process_video`` reads exactly
these three:
backend\eval\windowing.py:149:    Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob
reads exactly, against an
tests\test_job_worker_loop.py:70:    def process_video(self, video_id, video_path, *a, **k):
tests\test_job_worker_loop.py:135:        def process_video(self, video_id, video_path, *a,
**k):
tests\test_job_worker_loop.py:247:        def process_video(self, *a, **k):
backend\eval\serve_contrast.py:60:    est = rally_gate.estimate_play_axis(points)
backend\eval\served_gate_a.py:13:verification reflects what ``process_video`` actually hands to
Gemini.
backend\eval\served_gate_a.py:36:# TrajectoryHybridSegmenter.process_video reads for the served
fusion run).
backend\eval\served_gate_a.py:76:    # call the pure decision seams ? no process_video, no DB,
no provider ? exactly as the
backend\eval\served_gate_a.py:78:    seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport":
"badminton"})
tests\test_generate_highlights.py:97:    seg_instance.process_video.return_value = []          #
no rallies -> no compile/publish
tests\test_generate_highlights.py:124:    seg_instance.process_video.return_value = ["ok"]    #
non-Failure
tests\test_fusion_hybrid.py:54:        seg = FusionSegmenter(db, {"indexing": indexing or {}})
tests\test_fusion_hybrid.py:55:        res = seg.process_video("v1", "clip.mp4")
tests\test_fusion_hybrid.py:152:    seg = FusionSegmenter(MagicMock(), {"indexing": {}})
tests\test_gemini.py:42:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_gemini.py:81:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_gemini.py:108:    return segmenter.process_video(video_id, "dummy.mp4")
tests\test_gemini.py:185:        res = segmenter.process_video("vid_single_oversize",
"dummy.mp4")
tests\test_rally_gate.py:10:def test_estimate_play_axis_picks_larger_spread():
tests\test_rally_gate.py:12:    axis, net, span = rg.estimate_play_axis(pts)
tests\test_rally_gate.py:51:    est = rg.estimate_play_axis(pts)
tests\test_trajectory_hybrid_equivalence.py:56:        return db, seg.process_video("v1",
"clip.mp4")
tests\test_tracknet_hybrid.py:63:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:82:    assert seg.process_video("v1", "clip.mp4") == []
tests\test_tracknet_hybrid.py:95:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:110:    assert seg.process_video("v1", "clip.mp4") == []
```

```
tests\test_tracknet_hybrid.py:129:    seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:153:    res = seg.process_video("v1", "clip.mp4")
tests\test_tracknet_hybrid.py:181:    res = seg.process_video("v1", "clip.mp4")
tests\test_windowing.py:8:  - served_windowing_from_config reproduces process_video's knob reads
(overrides honoured).
tests\test_windowing.py:43:    """SERVED is single-sourced from IndexingConfig defaults ?
exactly what process_video reads."""
tests\test_yolo_hybrid.py:5:process_video OUTPUT is unchanged ? that is the regression guard for
the fusion-P1
tests\test_yolo_hybrid.py:89:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:113:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:156:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:225:    res = segmenter.process_video("v1", "dummy.mp4")
tests\test_yolo_hybrid.py:276:    segmenter.process_video("v1", "dummy.mp4")
tests\test_served_gate_a_regression.py:146:        est = rally_gate.estimate_play_axis(pts)
backend\pipeline\detectors\rally_gate.py:55:def estimate_play_axis(points) -> Tuple[str, float,
float]:
backend\pipeline\detectors\rally_gate.py:99:    `(axis, net, span)` (from `estimate_play_axis`)
so the O(P log P) estimate isn't redone
backend\pipeline\detectors\rally_gate.py:101:    axis, net, span = estimate if estimate is not
None else estimate_play_axis(points)
backend\pipeline\segmenters\motion.py:20:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\yolo_hybrid.py:57:    def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\gemini.py:35:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend\pipeline\segmenters\trajectory_hybrid.py:106:    def process_video(self, video_id: str,
video_path: str,  # type: ignore[override]
backend\pipeline\segmenters\base.py:24:    def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend\pipeline\segmenters\fusion_hybrid.py:39:class
FusionSegmenter(TrajectoryHybridSegmenter):
backend\pipeline\segmenters\fusion_hybrid.py:84:
min_crossings=0, estimate=self._axis_estimate(points))
backend\pipeline\segmenters\fusion_hybrid.py:100:    def _axis_estimate(self, points: List[Any])
-> tuple:
backend\pipeline\segmenters\fusion_hybrid.py:106:            self._axis_cache =
rally_gate.estimate_play_axis(points)
```

10. assistant (2026-06-18T07:52:39.273Z)

Now I need to check the call sites that loop over multiple videos with the same instance. The critical ones are the batch processors. Let me examine `tools/generate_highlights.py` (line 167 instantiates per-call inside a loop) and the eval scripts. Let me look at the loop structures.

11. user (2026-06-18T07:52:40.441Z)

```
90
91          cfg = load_config()
92          # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get).
93          # Logged by the segmenter itself once logging is configured below.
94          if skip_ai_handoff:
95              cfg.indexing.skip_ai_handoff = True
96          if wasb_stream:
97              cfg.indexing.wasb.stream_video = True
98          db = Database(db_path)
99          seg_cls = segmenter_registry.get(segmenter)
100         if seg_cls is None:
101             raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
102
103         # Combined batch log (local; published at the end).
104         fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
```

```
"%H:%M:%S")
105         if not logging.getLogger().handlers:
106             logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
107             for h in logging.getLogger().handlers:
108                 h.setFormatter(fmt)
109         batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
110         batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
111         batch_fh.setFormatter(fmt)
112         logging.getLogger().addHandler(batch_fh)
113
114         summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
115         logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
116                     len(video_paths), out_dir, segmenter, force)
117
118         for i, src in enumerate(video_paths, 1):
119             name = os.path.basename(src)
120             stem = os.path.splitext(name)[0]
121             reel_name = f"{stem} - Highlights.mp4"
122             dest_reel = os.path.join(out_dir, reel_name)
123             local_reel = os.path.join(local_work_dir, reel_name)
124             # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
125             # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
126             detect_src = detect_from_paths[i - 1] if detect_from_paths else src
127
128             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
129                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
130                 summary["skipped"].append(name)
131                 continue
132             if not os.path.exists(src) or not os.path.exists(detect_src):
133                 logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
134                                i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
135                 summary["failed"].append(name)
136                 continue
137
138             run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
139             run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
140             run_fh.setFormatter(fmt)
141             logging.getLogger().addHandler(run_fh)
142             staged_src: Optional[str] = None
143             try:
144                 logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
145                 # Localize the DETECTION source for WSL detectors (the WSL FS can't read
Drive/UNC
146                 # via /mnt). The stitch source (`src`) is read by ffmpeg directly during
compile, so
147                 # only the detector input ever needs staging.
148                 if needs_local:
149                     staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
150                     if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
151                         logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
152                         shutil.copy2(detect_src, staged_src)
153                     proc_detect = staged_src
154                 else:
155                     proc_detect = detect_src
156
157                 vid = compute_video_id(proc_detect)
158                 db.clear_video_data(vid)          # idempotent: no cross-run accumulation
159                 db.add_video(vid, proc_detect, "processed", [])
```

```
160                 if detect_src != src:
161                     logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
162                                 os.path.basename(detect_src), name)
163                 logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
164
165                 # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
166                 # main.py); cast keeps the type checker honest about that contract.
167                 result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid,
proc_detect)
168                 if isinstance(result, Failure):
169                     logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
170                     summary["failed"].append(name)
171                     continue
172
173                 segs = db.query_play_segments(vid, {})
174                 pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
175                 logger.info("rallies detected: %d", len(pairs))
176                 if not pairs:
177                     logger.warning("no rallies -> no reel")
178                     summary["no_rallies"].append(name)
179                     continue
```

12. assistant (2026-06-18T07:52:44.228Z)

Important: in `generate_highlights.py` line 167, a **fresh** `seg_cls(...)` instance is created for each video in the loop. So no instance reuse there. The cache is fresh per video. Let me check the eval scripts that loop, since they're the more likely candidates for instance reuse. Let me check `run_phase3_eval.py`, `run_process_and_stitch.py`, and the eval modules.

13. user (2026-06-18T07:52:46.111Z)

```
95
96          stages = batch.default_stages_for(args.segmenter, args.stage)
97
98          # Resolve the segmenter class up front (unless purely scoring).
99          segmenter = None
100         if not args.score_only:
101             seg_cls = segmenter_registry.get(args.segmenter)
102             if not seg_cls:
103                 print(f"ERROR: segmenter '{args.segmenter}' not found. "
104                       f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
105                 return 2
106             segmenter = seg_cls(db, cfg)
107
108         # Filter entries.
109         targets = []
110         for e in entries:
111             if args.only and e["video_id"] not in args.only:
112                 continue
113             targets.append(e)
114         if args.limit:
115             targets = targets[: args.limit]
116
117         per_video = []
118         skipped = []
119         print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
120
121         for e in targets:
122             vid = e["video_id"]
123             video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
124             csv_path = os.path.join(manifest_dir, e["csv"])
125
```

```
126             if not video_path or not os.path.exists(video_path):
127                 skipped.append((vid, "no video file"))
128                 print(f"[skip] {vid}: no video file ({e.get('local_path')})")
129                 continue
130             if not os.path.exists(csv_path):
131                 skipped.append((vid, "no GT csv"))
132                 print(f"[skip] {vid}: GT csv missing ({csv_path})")
133                 continue
134
135             video_id = get_file_md5(video_path)
136
137             # Segment unless score-only or already present (and not --reprocess).
138             if not args.score_only:
139                 already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
140                 if already and not args.reprocess:
141                     print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
142                 else:
143                     print(f"[proc] {vid}: segmenting with {args.segmenter}...")
144                     db.add_video(video_id, video_path, "processed", [])
145                     t0 = time.time()
146                     # process_video returns a Result (Success|Failure) for some segmenters
147                     # (e.g. the default `motion`); unwrap before len() so this doesn't crash
with
148                     # TypeError on a Success dataclass ? mirrors the unwrap in
backend/main.py.
149                     segs, failure = unwrap_or_failure(
150                         segmenter.process_video(video_id, video_path,
max_frames=args.max_frames))
151                     if failure is not None:
152                         print(f"        -> FAILED: {failure.message}")
153                         skipped.append((vid, f"segment failed: {failure.message}"))
154                         continue
155                     print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
156
157             gt = load_annotations(csv_path)
158             report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
159             print(format_report_text(report, show_failures=args.show_failures))
```

14. assistant (2026-06-18T07:52:58.352Z)

Found it. In `run_phase3_eval.py`:
- Line 106: `segmenter = seg_cls(db, cfg)` ? **ONE instance** created outside the loop.
- Line 121: `for e in targets:` ? loops over multiple videos.
- Line 150: `segmenter.process_video(video_id, video_path, ...)` ? **same instance reused across multiple videos**.

So instance reuse IS a real reachable path when `args.segmenter == "fusion_hybrid"`. This confirms the segmenter instance is shared across `process_video` calls for multiple videos.

Now the critical reachability question: for the stale-cache bug to actually fire, two conditions both need to hold across calls:
1. The fusion enrich path must run (net_crossings computed) ? requires `fusion_hybrid` segmenter, which is what `_axis_estimate` belongs to. Yes.
2. In call N+1, the new `points` list must get the **same `id()`** as the prior call's `points` list (which has been freed), AND the cache must still hold the old key.

Let me verify the exact lifecycle: after `process_video` call N returns, is `points` from call N freed before call N+1's `points` is allocated? `points` is a local in `process_video`, so it's freed when call N returns (unless referenced elsewhere). The cache stores `self._axis_cache_key = id(points_N)` and `self._axis_cache = estimate_N`. In call N+1, `points_{N+1} = runner.parse_trajectory_csv(...)`. If CPython reuses the freed address, `id(points_{N+1}) == id(points_N)`, the key matches, and the stale estimate is returned.

Let me check whether `points` is referenced anywhere that keeps it alive across calls, and confirm the `_axis_estimate` is reachable in the default `fusion_hybrid` config (it's called unconditionally in `_enrich_candidate_stats`).

Line 83-84: `gate_windows(points, ..., estimate=self._axis_estimate(points))` ? `_axis_estimate` is called **unconditionally** in the fusion enrich path, regardless of cue flags. So it runs on every fusion video. Good.

Now let me look at `estimate_play_axis` to understand what a "stale estimate" actually means for scoring.

15. user (2026-06-18T07:52:59.653Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21
22      from dataclasses import dataclass
23      from typing import List, Optional, Sequence, Tuple
24
25      Interval = Tuple[float, float]
26
27
28      @dataclass
29      class GatedWindow:
30          start: float
31          end: float
32          crossings: int
33          visible_points: int
34          kept: bool
35
36
37      def _spread(vals: Sequence[float]) -> float:
38          """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39          if len(vals) < 2:
40              return 0.0
41          s = sorted(vals)
42          lo = s[max(0, int(0.05 * (len(s) - 1)))]
43          hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44          return hi - lo
45
46
47      def _median(vals: Sequence[float]) -> float:
48          if not vals:
49              return 0.0
50          s = sorted(vals)
51          n = len(s)
52          return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
```

```


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06,
                     estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
        """Annotate each (start,end) window with its shuttle net-crossing count and a
        keep decision (crossings >= min_crossings). Deadband is a fraction of the play
        axis span. Net/axis are estimated once over the whole trajectory (stable).

        `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
        `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
        per window ? same result, avoids the per-window re-estimation."""
        axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
        deadband = deadband_frac * span
        out: List[GatedWindow] = []
        for (s, e) in windows:
            wp = _points_in(points, fps, s, e)
            c = count_net_crossings(wp, axis, net, deadband)
            vis = sum(1 for p in wp if p.visible)
            out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
                                   kept=c >= min_crossings))
        return out


    def apply_gate(points, windows: List[Interval], fps: float,
                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
        """Convenience: return only the windows that pass the crossing gate."""
```

```
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118
```

16. assistant (2026-06-18T07:53:09.944Z)

Now I understand the full impact. A stale `(axis, net, span)` from a *different* video would feed wrong `net`, `axis`, and `deadband` into `count_net_crossings`, producing a wrong `net_crossings` stat. With `min_crossings > 0` (Gate-A on), that could flip keep/drop decisions in `_select_for_handoff`.

But the candidate's confidence is 0.3 and severity low. My job is REACHABILITY. Let me now rigorously assess whether the `id()` reuse can actually occur. The key subtlety: **does the cache key from call N survive into call N+1?**

Critical detail I need to verify: in `process_video`, after the `window_stats` comprehension completes (line 230-233), is `points` still referenced? Yes ? `points` remains a live local in the `process_video` frame for the rest of the function (it's not reassigned). When call N **returns**, the frame is destroyed and `points_N` is freed. So between call N returning and call N+1 allocating `points_{N+1}`, the address is free and reusable.

But there's an even more important subtlety for the bug to be *observable*: the cache holds `self._axis_cache_key = id(points_N)`. For call N+1 to hit the stale cache, `id(points_{N+1})` must equal `id(points_N)`. This is the classic id-reuse trap. CPython does reuse freed addresses. However, this is **non-deterministic** and depends on allocation patterns ? but the candidate only claims it "can" happen, and reachability asks whether a real path *can* hit it.

Let me check the existing test to see if there's any guard/test already covering this scenario, and confirm the eval path actually uses `fusion_hybrid` with this reused instance.

17. user (2026-06-18T07:53:10.810Z)

```
1       """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3       Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4       keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5       substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6       persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7       """
8       from types import SimpleNamespace
9       from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17               "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19               "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22      def _points():
23          pts = []
24          for f in range(30, 120):       # window A frames: y flips 100<->900 around the net
(~500)
25              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26          for f in range(300, 360):      # window B frames: y == net level -> deadband -> no
crossings
27              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
```

```
        return pts


    def _runner():
        r = MagicMock()
        r.healthcheck.return_value = (True, "env ok")
        r.run_predict.return_value = "out.csv"
        r.parse_trajectory_csv.return_value = _points()
        r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
        return r


    def _run(indexing=None, provider_segments=None):
        provider = MagicMock()
        provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
        db = MagicMock()
        db.add_candidate_segment.side_effect = [101, 102, 103, 104]
        db.add_play_segment.return_value = 200

        with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)),
\
             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
             patch("os.environ.get", return_value="dummy_key"):
            preg.get.return_value = provider
            seg = FusionSegmenter(db, {"indexing": indexing or {}})
            res = seg.process_video("v1", "clip.mp4")
            return db, provider, res


    def _stats_calls(db):
        """Persisted stats dicts, in call order."""
        return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]


    def test_net_crossings_persisted_and_person_off_by_default():
        db, _provider, _res = _run()
        stats = _stats_calls(db)
        assert len(stats) == 2                              # both windows persisted
(superset)
        assert all("net_crossings" in s for s in stats)
        # Window A (rally) crosses many times; window B (held) zero.
        assert stats[0]["net_crossings"] >= 2
        assert stats[1]["net_crossings"] == 0.0
        # Person cue OFF by default -> NONE of the person features present.
        assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
        # The 4 base motion keys are still there.
        assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
                   for s in stats)


    def test_persisted_candidates_are_superset_of_substrate_windows():
        db, _provider, _res = _run()
        spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
        assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly the substrate windows,
unfiltered
```

```
83
84
85      def test_default_hands_off_all_windows():
86          _db, provider, _res = _run()                        # min_crossings default 0 -> no
gating
87          assert provider.analyze_video.call_count == 2
88
89
90      def test_gate_a_drops_low_crossing_window_from_handoff_only():
91          db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92          # Persistence is still the full superset (offline substrate intact)...
93          assert len(_stats_calls(db)) == 2
94          # ...but only the high-crossing window A reaches the AI handoff.
95          assert provider.analyze_video.call_count == 1
96
97
98      def test_gate_a_never_empties_a_video():
99          """NEVER-EMPTY safeguard: if the crossings threshold would drop EVERY window, the
gate
100         falls back to the full ungated set rather than hand off zero rallies. Here
min_crossings
101         is set absurdly high so neither window clears it ? yet both still reach the
handoff."""
102         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 999}})
103         assert len(_stats_calls(db)) == 2                   # superset persisted, untouched
104         # Gating would zero the video ? safeguard restores the ungated windows (both hand
off).
105         assert provider.analyze_video.call_count == 2
106
107
108     def test_person_cue_adds_features_when_enabled():
109         # Two in-court players, each shifting between two sampled frames ? exercises the
REAL
110         # glue (in-court counts -> players_in_court; track displacement ->
mean_player_motion),
111         # not just key-presence.
112         fake_pd = MagicMock()
113         fake_pd.detections_per_frame.return_value = {
114             "frames": {
115                 30: [(1, (10, 10, 30, 40)), (2, (60, 10, 80, 40))],
116                 33: [(1, (16, 10, 36, 40)), (2, (66, 10, 86, 40))],
117             },
118             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
119         }
120         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
121             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
122         stats = _stats_calls(db)
123         assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
124         assert all("net_crossings" in s for s in stats)
125         assert stats[0]["players_in_court"] == 2.0          # 2 in-court players every
sampled frame
126         assert stats[0]["mean_player_motion"] > 0.0         # both tracks moved -> non-zero
motion
127         fake_pd.detections_per_frame.assert_called()        # the person detector WAS used
128
129
130     def test_gate_b_drops_low_player_window_from_handoff():
131         """Gate B: with the person cue on and min_players=2, a window whose in-court player
132         count is below 2 is dropped from the handoff ? but still persisted (superset)."""
133         def _side(video_path, start_f, end_f, frame_skip):
134             if start_f >= 300:        # window B [10,12]s -> start frame ~300: only 1 player
135                 frames = {start_f: [(1, (10, 10, 30, 40))]}
136             else:                     # window A [1,4]s -> start frame ~30: 2 players
137                 frames = {start_f: [(1, (10, 10, 30, 40)), (2, (60, 10, 80, 40))]}
```

```
138             return {"frames": frames, "fps": 30.0, "frame_width": 1000.0, "frame_height":
1000.0}
139
140         fake_pd = MagicMock()
141         fake_pd.detections_per_frame.side_effect = _side
142         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
143             db, provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True},
"min_players": 2}})
144         stats = _stats_calls(db)
145         assert len(stats) == 2                                # superset persisted (both)
146         assert provider.analyze_video.call_count == 1        # only window A (>=2 players)
handed off
147         assert stats[0]["players_in_court"] >= 2 and stats[1]["players_in_court"] < 2
148
149
150     def test_build_runner_selects_substrate():
151         """_build_runner picks the WASB or TrackNet runner by indexing.fusion.substrate."""
152         seg = FusionSegmenter(MagicMock(), {"indexing": {}})
153         with patch("backend.pipeline.segmenters.fusion_hybrid.WasbRunner") as W, \
154              patch("backend.pipeline.segmenters.fusion_hybrid.WasbConfig") as WC, \
155              patch("backend.pipeline.segmenters.fusion_hybrid.TrackNetRunner") as T, \
156              patch("backend.pipeline.segmenters.fusion_hybrid.TrackNetConfig") as TC:
157             WC.from_indexing_cfg.return_value = MagicMock(weights_path="w")
158             TC.from_indexing_cfg.return_value = MagicMock(weights_path="t", queue_length=5)
159             _runner_w, extras_w = seg._build_runner({"fusion": {"substrate": "wasb"}})
160             _runner_t, extras_t = seg._build_runner({"fusion": {"substrate": "tracknet"}})
161             _runner_d, extras_d = seg._build_runner({})       # default -> wasb
162         assert W.called and extras_w["fusion_substrate"] == "wasb"
163         assert T.called and extras_t["fusion_substrate"] == "tracknet"
164         assert extras_t["queue_length"] == 5
165         assert extras_d["fusion_substrate"] == "wasb"
166
```

18. assistant (2026-06-18T07:53:21.817Z)

The test always creates a fresh `FusionSegmenter` per `_run()` call, so no test exercises instance reuse across videos. The bug is not already guarded or tested.

Now let me carefully assess REACHABILITY. The path that reuses one instance across videos is `run_phase3_eval.py` (line 106 creates one instance, line 150 calls `process_video` in a loop). This is an offline eval/dev script, not production serving, but it IS a real code path.

However, I need to be rigorous about whether `id()` reuse can *actually* produce a stale read here. There's a subtle but decisive point. Within `_axis_estimate`:

```python
key = id(points)
if self._axis_cache_key != key or self._axis_cache is None:
    self._axis_cache_key = key
    self._axis_cache = rally_gate.estimate_play_axis(points)
```

For the stale read in call N+1, we need `id(points_{N+1}) == self._axis_cache_key` where `self._axis_cache_key == id(points_N)`.

Critical insight about lifetime overlap: `points_N` (call N's trajectory) is freed when call N's `process_video` frame is destroyed (on return). In call N+1, `points_{N+1}` is allocated *after* call N already returned. So `points_N` is dead before `points_{N+1}` is created ? the addresses can coincide. This is the genuine id-reuse trap. The cache value `self._axis_cache` (the estimate tuple) is held alive by `self`, so it persists across calls.

So the bug is genuinely reachable in principle. But let me determine how *likely* / deterministic. The candidate itself rates confidence 0.3 and severity low, acknowledging "CPython readily reuses freed list addresses."

Let me verify whether the eval path realistically uses `fusion_hybrid`. Let me check `batch.default_stages_for` and whether fusion is a supported eval segmenter, and check `run_process_and_stitch.py` for instance reuse too.

19. user (2026-06-18T07:53:23.370Z)

```
55              segments = db.query_play_segments(video_id, {})
56              if len(segments) > 0:
57                  print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
58                  choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
59                  if choice == '1':
60                      print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
61                      cache_hit = True
62                  else:
63                      print("[CACHE] Ignoring cache...", flush=True)
64              else:
65                  print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
66
67          if not cache_hit:
68              print("[CACHE] Initializing clean indexing...", flush=True)
69              db.clear_video_data(video_id)
70
71              print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
72              db.add_video(video_id, video_path, "processed", [])
73
74              print("Running segmenter & indexing...", flush=True)
75              seg_name = cfg.indexing.default_segmenter
76              segmenter_cls = segmenter_registry.get(seg_name)
77              if not segmenter_cls:
78                  print(f"[FAIL] Segmenter '{seg_name}' not found.")
79                  return
80              print(f"Using segmenter: {seg_name}", flush=True)
81              segmenter = segmenter_cls(db, cfg)
82              # process_video returns a Result (Success|Failure) for some segmenters (e.g.
`motion`);
83              # unwrap before len()/truthiness so this doesn't crash with TypeError on a
Success
84              # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
85              segments, failure = unwrap_or_failure(segmenter.process_video(video_id,
video_path))
86              if failure is not None:
87                  print(f"[FAIL] Segmentation failed: {failure.message}")
88                  return
89              print(f"Successfully indexed {len(segments)} play segments.", flush=True)
90
91          if not segments:
92              print("No play segments detected. Cannot compile highlights.")
93              return
94
95
96          end_padding = cfg.stitching.end_padding
97          begin_padding = cfg.stitching.begin_padding
98          min_shot_count = cfg.stitching.min_shot_count
99
```

20. assistant (2026-06-18T07:53:26.872Z)

`run_process_and_stitch.py` processes a single video per invocation (no multi-video loop reusing the instance), so it's not a reuse path. The relevant reuse path is `run_phase3_eval.py` (and

`backend/main.py` creates a fresh instance per request ? let me confirm).

21. user (2026-06-18T07:53:28.097Z)

```
165
166             # 2. Run segmenter & indexer
167             logger.info("[API] Video passed checks. Segmenting and indexing...")
168             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
169
170             segmenter_cls = segmenter_registry.get(seg_name)
171             if not segmenter_cls:
172                 # Submit-time validation already 400s unknown names; this is the defensive
173                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
174                 available = list(segmenter_registry.list_available().keys())
175                 response = {
176                     "video_id": video_id,
177                     "status": "failed",
178                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
179                     "results": [r.model_dump() for r in val_results],
180                     "play_segments": [],
181                 }
182                 return response
183
184             logger.info(f"[API] Using segmenter: {seg_name}")
185             notify(f"segmenting ({seg_name})")
186             segmenter_actual = seg_name
187             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
188             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
189             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
190             segmenter = segmenter_cls(db_instance, global_config)
191             try:
192                 result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
193             except Exception:
194                 # A raising segmenter must not leave half-written rows: restore the pre-run
195                 # state (same destroy-after-confirm contract as the Failure branch), then
let
196                 # the job worker record the failure.
197                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
198                 raise
199             segmentation_ran = True
```

22. assistant (2026-06-18T07:53:45.637Z)

Confirmed: `backend/main.py` (line 190) creates a **fresh** `segmenter` instance per request (inside the per-video processing path). So production serving never reuses the instance ? the cache is always fresh there. No production reachability.

The only multi-video-loop reuse path is `scripts/run_phase3_eval.py` (offline eval/dev tooling). Let me reason about the actual reachability there with full rigor.

For the stale read to fire in `run_phase3_eval.py`:
1. `args.segmenter` must be `fusion_hybrid`. The script accepts an arbitrary `--segmenter` arg, so this is selectable. Reachable.
2. The loop processes ?2 videos with that one instance. Reachable (multi-target eval).
3. In video N+1, `id(points_{N+1}) == id(points_N)` (the freed address is reused).

Point 3 is the crux. This is genuinely non-deterministic but it CAN happen. However, there's a strong mitigating factor I should weigh carefully: `points` is a Python `list`. CPython's `id()` reuse for the *list object header* requires the same-size object allocator (pymalloc / obmalloc)

to hand back the exact same address. For a `list` object, the list header is a fixed-size `PyListObject` allocated from pymalloc's pools. When a list of the same allocation class is freed and a new list is created, pymalloc's LIFO free-list behavior makes address reuse **quite likely** in practice ? this is the well-documented "id reuse" gotcha (e.g. `id(a) == id(b)` for short-lived objects).

But ? decisive subtlety ? in `run_phase3_eval.py`, between call N returning and call N+1's `points = runner.parse_trajectory_csv(...)`, there is substantial intervening work in call N+1's `process_video` before `points` is allocated: `get_video_duration`, `_probe_fps_width` (opens cv2.VideoCapture), `_build_runner`, `runner.healthcheck()`, `runner.run_predict()` (runs the whole WASB/TrackNet inference, allocating/freeing huge amounts), and `runner.parse_trajectory_csv()` itself (which allocates many `TrajectoryPoint` objects before/while building the list). All of that allocation churn happens between freeing `points_N` and creating `points_{N+1}`. After that much allocation activity, the probability that the new list object lands at *exactly* the freed `points_N` address is effectively negligible ? pymalloc's free list for that size class will have been disturbed many times over.

So while the candidate's mechanism is theoretically sound (id reuse is real), the concrete path has so much intervening allocation that the same-address coincidence is practically unreachable in this specific call sequence. This is not a deterministic or even plausibly-triggerable defect on a real input ? it's a latent fragility (relying on `id()` as a cache key is unsound style) rather than a reachable defect.

Let me double-check there's no *other* reuse pattern where two `_axis_estimate` calls happen close together with minimal intervening allocation ? e.g., a harness that calls the enrich seam repeatedly on the same instance with different `points`. Let me check the eval modules `served_gate_a.py` and `fusion_golden.py`.

23. user (2026-06-18T07:53:46.744Z)

```
60                                  min_crossings: int) -> List[Interval]:
61          """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
return the
62          windows that would reach the AI handoff (i.e. the served rally candidates).
63
64          Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
(computes
65          ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
production
66          ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
is OFF.
67          """
68          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
69          windows = TrackNetRunner.trajectory_to_action_windows(
70              points, fps=fps, frame_width=frame_width,
71              velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
72              min_window_duration=PROD_MIN_WINDOW,
73          )
74          idx_cfg = _served_idx_cfg(min_crossings)
75          # Construct the real served segmenter (db=None, config carries the indexing block).
We only
76          # call the pure decision seams ? no process_video, no DB, no provider ? exactly as
the
77          # registry constructs it for `name`/`description`.
78          seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
79          # The served per-window stats (incl. net_crossings) via the production enrichment
hook.
80          window_stats = [
81              seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
82                                          idx_cfg=idx_cfg)
83              for cw in windows
84          ]
85          keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
86          return [(windows[i]["start"], windows[i]["end"]) for i in keep]
87
```

```
88
89      @dataclass
90      class ServedVideoResult:
91          name: str
92          num_gt: int
93          on: Score                # Gate-A ON  (min_crossings=2)
94          off: Score               # Gate-A OFF (min_crossings=0, control)
95          seg_ratio_on: float      # |preds| / |gts| with Gate-A ON  (over-segmentation)
96          seg_ratio_off: float
97          n_handoff_on: int
98          n_handoff_off: int
99
```

24. assistant (2026-06-18T07:53:51.734Z)

This `served_gate_a.py` function creates a **fresh** `FusionSegmenter` per call (line 78), and within the function all enrich calls share the **same** `points` (bound at line 68, alive through the comprehension). So no stale-cache issue here either ? within one `points` lifetime the cache is correct, which is the intended behavior. If `process_served_gate_a` is called in a loop over videos, each call constructs a new `seg`, so no instance reuse.

Let me check `fusion_golden.py` to be thorough about whether any harness reuses one `FusionSegmenter` instance across multiple `points` lists in a tight loop (which would be the realistic trigger).

25. user (2026-06-18T07:53:53.228Z)

```
backend\eval\rally_seg_eval.py:129:    NEVER-EMPTY safeguard (mirrors
``FusionSegmenter._select_for_handoff``): if the gate would
backend\eval\served_gate_a.py:1:"""SERVED Gate-A verification: run the *production*
``FusionSegmenter`` decision seams
backend\eval\served_gate_a.py:10:net-crossing count is computed by
``FusionSegmenter._enrich_candidate_stats``, and the gate is
backend\eval\served_gate_a.py:11:``FusionSegmenter._select_for_handoff``. THIS module drives
exactly those production methods ?
backend\eval\served_gate_a.py:30:from backend.pipeline.segmenters.fusion_hybrid import
FusionSegmenter
backend\eval\served_gate_a.py:43:    """The ``indexing`` config dict the served FusionSegmenter
seams read ? mirrors
backend\eval\served_gate_a.py:61:    """Run the PRODUCTION FusionSegmenter decision path on a
cached trajectory and return the
backend\eval\served_gate_a.py:64:    Uses the real ``FusionSegmenter`` seam methods ?
``_enrich_candidate_stats`` (computes
backend\eval\served_gate_a.py:78:    seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport":
"badminton"})
backend\eval\served_gate_a.py:81:        seg._enrich_candidate_stats(cw, points, fps,
frame_width, video_path="",
backend\eval\served_gate_a.py:159:        f"=== SERVED Gate-A verification (production
FusionSegmenter path, IoU>={iou}) ===",
backend\eval\served_gate_a.py:190:        description="Verify served Gate-A on real footage via
the production FusionSegmenter path.")
```

26. assistant (2026-06-18T07:53:55.966Z)

Let me confirm whether `process_served_gate_a` (the function at line 60) is invoked in a loop with a single shared instance ? checking the rest of `served_gate_a.py`.

27. user (2026-06-18T07:53:56.736Z)

```
100         @property
101         def delta_f1(self) -> float:
102             return self.on.f1 - self.off.f1
103
104
```

```
105     def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
106                        on_crossings: int = 2) -> ServedVideoResult:
107         """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
windows for
108         one golden video against its golden labels."""
109         fps, fw = float(spec["fps"]), float(spec["frame_width"])
110         traj, labels = str(spec["trajectory"]), str(spec["labels"])
111         gts = load_golden_gts(labels)
112         preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
113         preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
114         return ServedVideoResult(
115             name=str(spec["name"]),
116             num_gt=len(gts),
117             on=score(preds_on, gts, iou),
118             off=score(preds_off, gts, iou),
119             seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
120             seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
121             n_handoff_on=len(preds_on),
122             n_handoff_off=len(preds_off),
123         )
124
125
126     def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
127                           on_crossings: int = 2) -> List[ServedVideoResult]:
128         return [evaluate_video(s, iou, on_crossings) for s in specs]
129
130
131     def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
132         """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
133         n = len(results)
134
135         def _mean(getter) -> float:
136             return sum(getter(r) for r in results) / n if n else 0.0
137
138         deltas = [r.delta_f1 for r in results]
139         wins = sum(1 for d in deltas if d > 1e-9)
140         losses = sum(1 for d in deltas if d < -1e-9)
141         return {
142             "num_videos": n,
143             "mean_f1_on": _mean(lambda r: r.on.f1),
144             "mean_f1_off": _mean(lambda r: r.off.f1),
145             "mean_delta_f1": _mean(lambda r: r.delta_f1),
146             "mean_precision_on": _mean(lambda r: r.on.precision),
147             "mean_precision_off": _mean(lambda r: r.off.precision),
148             "mean_recall_on": _mean(lambda r: r.on.recall),
149             "mean_recall_off": _mean(lambda r: r.off.recall),
150             "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
151             "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
152             "wins": wins,
153             "losses": losses,
154         }
155
156
157     def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
str:
158         lines = [
159             f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
===",
```

28. assistant (2026-06-18T07:54:10.665Z)

This confirms the eval path constructs a **fresh** `FusionSegmenter` inside `served_handoff_windows` for every video (and even calls it twice ? ON and OFF ? each with its own fresh instance). So no instance reuse here. The only instance-reuse-across-videos path in the whole codebase is `run_phase3_eval.py` line 106 ? 150.

I now have everything needed for a rigorous verdict.

Verdict on REACHABILITY

The mechanism is real but the concrete path is practically unreachable.

What I confirmed:
- **`_axis_estimate` is unconditionally reached** in the fusion enrich path (`fusion_hybrid.py:84`), and `_select_for_handoff` can flip keep/drop when `min_crossings > 0`, so a stale estimate *would* mis-score ? the consequence is real.
- **Within a single `process_video`, the cache is correct**: `points` (`trajectory_hybrid.py:202`) stays alive through the enrich comprehension (lines 230-233), so `id(points)` is stable. Intended behavior, no bug.
- **Production serving never reuses the instance**: `backend/main.py:190` and `tools/generate_highlights.py:167` both create a fresh `FusionSegmenter` per video, so the cache always starts empty. No production reachability.
- **The only multi-video loop that reuses one instance** is `scripts/run_phase3_eval.py` (instance at line 106, `process_video` in a loop at line 150) ? an offline eval/dev script, only when `--segmenter fusion_hybrid`.

Why it's still not realistically triggerable even on that path: for a stale read, video N+1's freshly-parsed `points` list must land at the **exact** freed address of video N's `points`. Between freeing `points_N` (call N returns) and allocating `points_{N+1}`, call N+1 runs `get_video_duration`, `_probe_fps_width`, `_build_runner`, `healthcheck`, the **full detector inference** (`run_predict`), and `parse_trajectory_csv` (which itself allocates all the `TrajectoryPoint` objects and grows the list). That volume of intervening allocation churn makes pymalloc handing back precisely the old list-header address negligibly unlikely. The id-reuse gotcha needs near-back-to-back alloc/free with no churn between; this path has the opposite.

So this is a latent code-smell (using `id()` as a cross-lifetime cache key is unsound and the proposed content-fingerprint/per-video-reset fix is good hygiene), not a defect reachable by any realistic input on any real call path. I'm setting `real=false` on the reachability lens.

29. user (2026-06-18T07:54:21.592Z)

Structured output provided successfully

30. assistant (2026-06-18T07:54:30.644Z)

I've completed the reachability analysis. Verdict: **not a real, reachable defect** (`real=false`).

Summary of findings:
- The stale-cache mechanism in `backend/pipeline/segmenters/fusion_hybrid.py:103-106` (`_axis_estimate`, keyed on `id(points)`) is only correct within a single `process_video` call, where `points` (`trajectory_hybrid.py:202`) stays alive through the enrich loop. That is the intended, working case.
- For a stale read, the `FusionSegmenter` instance must be reused across videos. Production never does this: `backend/main.py:190`, `tools/generate_highlights.py:167`, and `backend/eval/served_gate_a.py:78` all construct a fresh instance per video.
- The only instance-reuse-across-videos loop is `scripts/run_phase3_eval.py` (instance at line 106, `process_video` looped at line 150, only with `--segmenter fusion_hybrid`).
- Even there, the `id()` collision is not realistically triggerable: between freeing call N's `points` and allocating call N+1's, the code runs the full detector inference (`run_predict`) plus `parse_trajectory_csv` (allocating all `TrajectoryPoint`s and growing the list). That allocation churn makes pymalloc returning the exact freed list-header address negligibly unlikely.

It's a genuine code-smell (using `id()` as a cross-lifetime cache key is unsound; the proposed fix is good hygiene), but not a reachable bug.